

Fixing allocator usage for operator+ (basic_string)

Paper number	P1165R0
Reply-to	Tim Song < rs2740@gmail.com >
Audience	LWG

Abstract

Allocator propagation for `basic_string`'s `operator+` is haphazard, inconsistent, and a source of implementation divergence. Let's make it consistent.

Introduction

Let `lhs` and `rhs` be two `basic_string` lvalues with a `char value_type`. The following table shows the allocator used for the result for each of the 12 `operator+` overloads, in the working paper and three major implementations:

Expression	WP	libstdc++ (trunk)	libc++ (trunk)	MSVC STL (19.00.23506)
<code>lhs + rhs</code>	SOCCC on SOCCC on lhs	SOCCC on lhs	lhs (no SOCCC)	default- constructed
<code>lhs + std::move(rhs)</code>	rhs	rhs	rhs	rhs
<code>std::move(lhs) + rhs</code>	lhs	lhs	lhs	lhs
<code>std::move(lhs) + std::move(rhs)</code>	lhs "or equivalently" rhs	lhs	lhs	lhs
<code>lhs + "str"</code>	default- constructed	SOCCC on lhs	lhs	default- constructed
<code>lhs + 'c'</code>	default- constructed	SOCCC on lhs	lhs	default- constructed
<code>std::move(lhs) + "str"</code>	lhs	lhs	lhs	lhs
<code>std::move(lhs) + 'c'</code>	lhs	lhs	lhs	lhs
<code>"str" + rhs</code>	default- constructed	default- constructed	rhs	default- constructed

Expression	WP	libstdc++ (trunk)	libc++ (trunk)	MSVC STL (19.00.23506)
'c' + rhs	default-constructed	default-constructed	rhs	default-constructed
"str" + std::move(rhs)	rhs	rhs	rhs	rhs
'c' + std::move(rhs)	rhs	rhs	rhs	rhs

(SOCCC == [select_on_container_copy_construction](#). The ~~struck through~~ text is proposed to be removed by P1148R0.)

This is haphazard, inconsistent, and a source of implementation divergence.

Discussion

In the discussion of [LWG2402](#), Pablo Halpern explained the patterns we use for constructors of allocator-aware containers (modifications mine):

- Every constructor needs a version with and without an allocator argument (possibly through the use of default arguments).
- Every constructor except the copy [and move] constructor[s] for which an allocator is not provided uses a default-constructed allocator.

Copy constructors obtain the allocator to use via `select_on_container_copy_construction`; move constructors obtain the allocator to use by move constructing from the source's allocator (which is equivalent to copy after [LWG2593](#)).

`operator+` constructs a new string, but it is not a copy or move constructor any more than `basic_string(const basic_string&, size_t, size_t, const Allocator&)`. Its operands are simply sources of characters. It should therefore consistently use a default-constructed allocator.

As a binary operator, it is not possible to add an allocator argument to `operator+`. Designing a string concatenation API with allocator support is beyond the scope of this paper. Users desiring to control the allocator used for the result should therefore use the member functions of `basic_string` directly.

Proposed wording

This wording is relative to [N4762](#).

Replace `[string.op+]` in its entirety with the following:

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const basic_string<charT, traits, Allocator>& lhs,
```

```

        const basic_string<charT, traits, Allocator>& rhs);

template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const basic_string<charT, traits, Allocator>& lhs, const charT*

```

¹ *Effects*: Equivalent to:

```

    basic_string<charT, traits, Allocator> r(lhs, Allocator());
    r.append(rhs);
    return r;

```

```

template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(basic_string<charT, traits, Allocator>&& lhs,
                  const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(basic_string<charT, traits, Allocator>&& lhs, const charT* rhs)

```



² *Effects*: Equivalent to:

```

    basic_string<charT, traits, Allocator> r(std::move(lhs), Allocator());
    r.append(rhs);
    return r;

```

```

template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(basic_string<charT, traits, Allocator>&& lhs,
                  basic_string<charT, traits, Allocator>&& rhs);

```

³ *Effects*: Equivalent to:

```

    basic_string<charT, traits, Allocator> r(std::move(lhs), Allocator());
    r.append(rhs);
    return r;

```

except that both lhs and rhs are left in valid but unspecified states.

Drafting note: The intent is to allow the implementation to move from either operand. — *end drafting note*

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const basic_string<charT, traits, Allocator>& lhs,
                   basic_string<charT, traits, Allocator>&& rhs);
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const charT* lhs, basic_string<charT, traits, Allocator>&& rhs)
```

⁴ Effects: Equivalent to:

```
basic_string<charT, traits, Allocator> r(std::move(rhs), Allocator());
r.insert(0, lhs);
return r;
```

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const charT* lhs, const basic_string<charT, traits, Allocator>&
```

⁵ Effects: Equivalent to:

```
basic_string<charT, traits, Allocator> r = lhs;
r.append(rhs);
return r;
```

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(charT lhs, const basic_string<charT, traits, Allocator>& rhs);
```

⁶ Effects: Equivalent to:

```
basic_string<charT, traits, Allocator> r(1, lhs);
r.append(rhs);
return r;
```

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(charT lhs, basic_string<charT, traits, Allocator>&& rhs);
```

⁷ *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r(std::move(rhs), Allocator());
r.insert(r.begin(), lhs);
return r;
```

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const basic_string<charT, traits, Allocator>& lhs, charT rhs);
```



⁸ *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r(lhs, Allocator());
r.push_back(rhs);
return r;
```

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(basic_string<charT, traits, Allocator>&& lhs, charT rhs);
```

⁹ *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r(std::move(lhs), Allocator());
r.push_back(rhs);
return r;
```